

EXPERIMENT 4

Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.

CODE 1:

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int prime[1000], i, j, n;

    // Prompt user for input
    printf("\nIn order to find prime numbers from 1 to n, enter the value of n: ");
    scanf("%d", &n);

    // Initialize all numbers as prime
    for (i = 1; i <= n; i++)
    {
        prime[i] = 1;
    }

    // 1 is not a prime number
    prime[1] = 0;

    // Sieve of Eratosthenes with parallelization
    for (i = 2; i * i <= n; i++)
    {
        #pragma omp parallel for
        for (j = i * i; j <= n; j = j + i)
        {
            if (prime[j] == 1)
            {
                prime[j] = 0;
            }
        }
    }

    // Print prime numbers
    printf("\nPrime numbers from 1 to %d are:\n", n);

    for (i = 2; i <= n; i++)
    {
        if (prime[i] == 1)
        {
            printf("%d\t", i);
        }
    }
}
```

```

    }
}

printf("\n");

return 0;
}

```

OUTPUT:

\$ gcc -fopenmp prime.c

\$./a.out

In order to find prime numbers from 1 to n, enter the value of n: 100

Prime numbers from 1 to 100 are:

```

2 3 5 7 11 13 17 19 23 29 31
37 41 43 47 53 59 61 67 71 73
79 83 89 97

```

CODE 2:

```

#include <stdio.h>
#include <omp.h>

int main()
{
    int prime[1000], i, j, n;

    // Prompt user for input
    printf("\nIn order to find prime numbers from 1 to n, enter the value of
n: ");
    scanf("%d", &n);

    // Initialize all numbers as prime
    for (i = 1; i <= n; i++)
    {
        prime[i] = 1;
    }

    // 1 is not a prime number
    prime[1] = 0;

    // Sieve of Eratosthenes with parallelization
    for (i = 2; i * i <= n; i++)

```

```

{
    #pragma omp parallel for
    for (j = i * i; j <= n; j = j + i)
    {
        if (prime[j] == 1)
        {
            prime[j] = 0;
        }
    }
}

// Print prime numbers
printf("\nPrime numbers from 1 to %d are:\n", n);

for (i = 2; i <= n; i++)
{
    if (prime[i] == 1)
    {
        printf("%d\t", i);
    }
}

double start, end;

// Sequential
start = omp_get_wtime();

prime[i];

end = omp_get_wtime();

printf("\nSequential Time: %f seconds\n", end - start);

// Parallel
start = omp_get_wtime();

prime[i];

end = omp_get_wtime();

printf("Parallel Time: %f seconds\n", end - start);

printf("\n");

return 0;
}

```

OUTPUT:

```
ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc -fopenmp prime.c
```

```
ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
```

In order to find prime numbers from 1 to n, enter the value of n: 100

Prime numbers from 1 to 100 are:

2 3 5 7 11 13 17 19 23 29 31

37 41 43 47 53 59 61 67 71 73

79 83 89 97

Sequential Time: 0.000001 seconds

Parallel Time: 0.000000 seconds